

Template Method e Memento

Padrões de Projeto - Aula #11

Prof. Dr. Carlos Melo

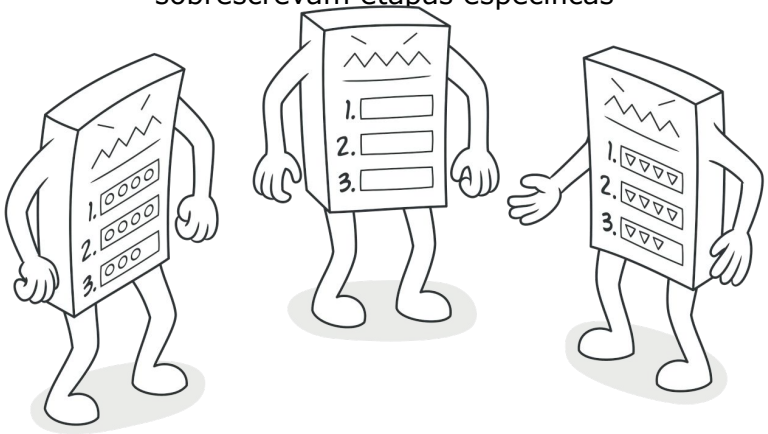
carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Padrões Comportamentais (parte #4)

Template Method

Define o esqueleto de um algoritmo em uma superclasse, permitindo que as subclasses sobrescrevam etapas específicas



Memento

Permite guardar e restaurar estados anteriores de um objeto, preservando seu encapsulamento



- Template Method sobrescreve etapas sem alterar a estrutura da operação
- Memento trata da captura e restauração de estados anteriores

carlos.melo@upe.br

Objetivos de aprendizagem

- Identificar comportamentos e transições que dependem do estado de um objeto
- Implementar *State* e distingui-lo de *Strategy* pela intenção
- Implementar a captura e a restauração do estado de um editor
- Reconhecer quando cada padrão justifica a estrutura adicional

Memento - O Problema

- No editor de texto acompanharemos o texto e a posição do cursor

Momento	Texto	Cursor
Antes da edição	"Olá"	3
Depois de inserir " mundo"	"Olá mundo"	9
Depois de desfazer	"Olá"	3

- O cursor indica a posição de inserção:
  - `0` representa o início do texto
- Para desfazer, precisamos recuperar os valores anteriores
- Se o histórico copiar diretamente os atributos do editor, ficará dependente da sua estrutura interna

Memento - ideia e intenção

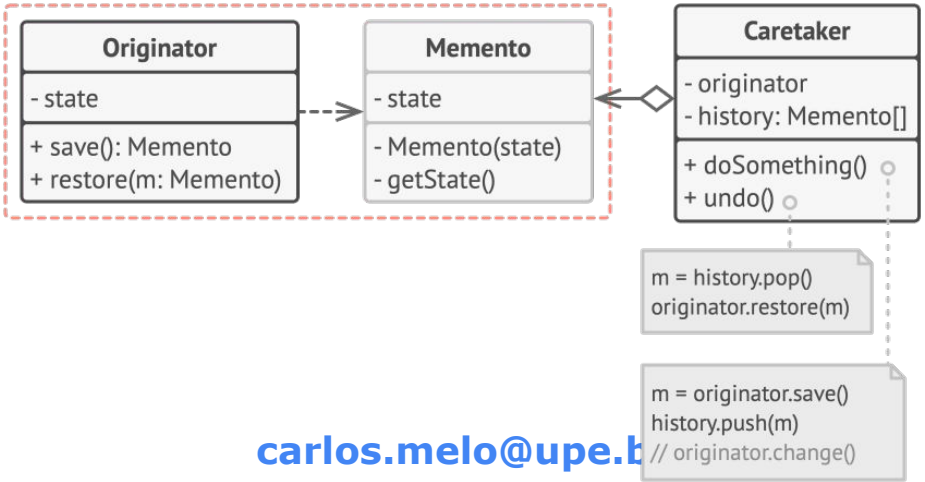
"Permitir salvar e restaurar o estado anterior de um objeto sem revelar os detalhes de sua implementação"

- Ideia central:
  - O próprio objeto cria um retrato do seu estado, chamado Memento
  - Outro objeto guarda esse retrato e pode solicitar sua restauração, sem precisar interpretar os dados armazenados
- As regras de publicação ficam nas classes Rascunho, Moderacao e Publicado

Memento - Estrutura baseada em classes aninhadas

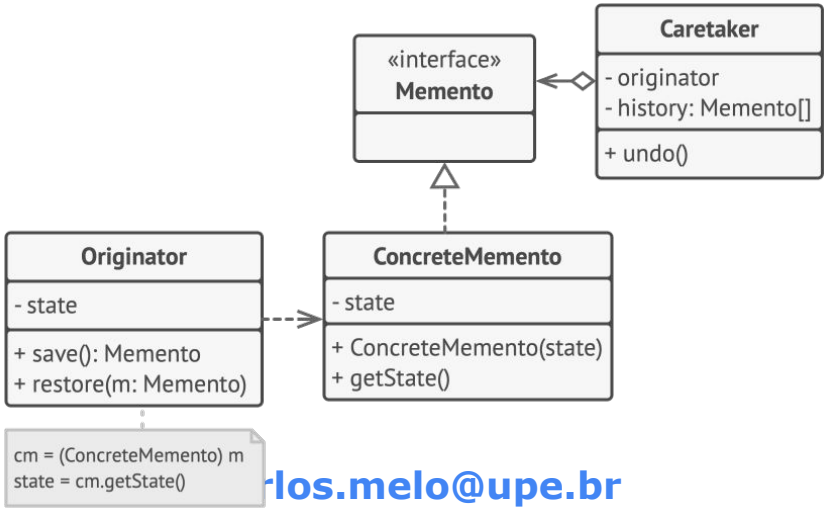
- A **Originadora** produz um retrato do próprio estado e sabe restaurá-lo
- A **Cuidadora** organiza o histórico desses retratos

1. Antes da edição, o histórico solicita um retrato ao editor
2. O editor cria um Memento com os valores atuais
3. A edição modifica o editor e o retrato permanece estável
4. Ao desfazer, o histórico devolve o retrato ao editor, que restaura seus valores

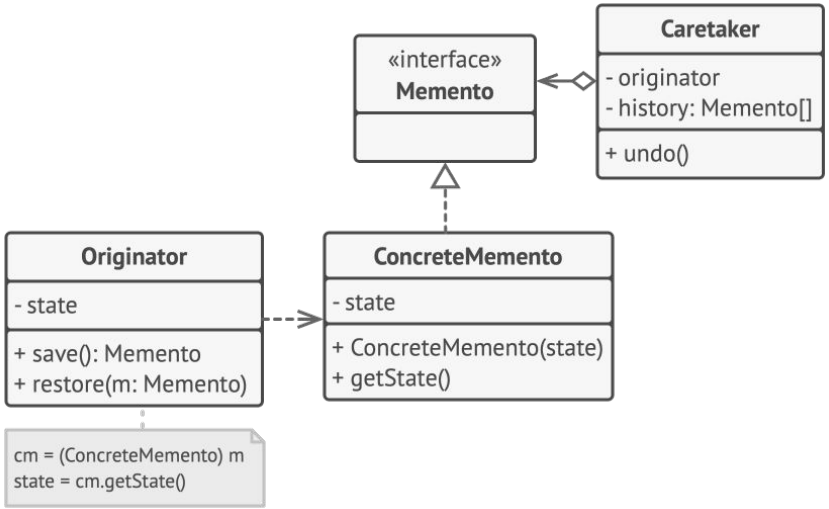


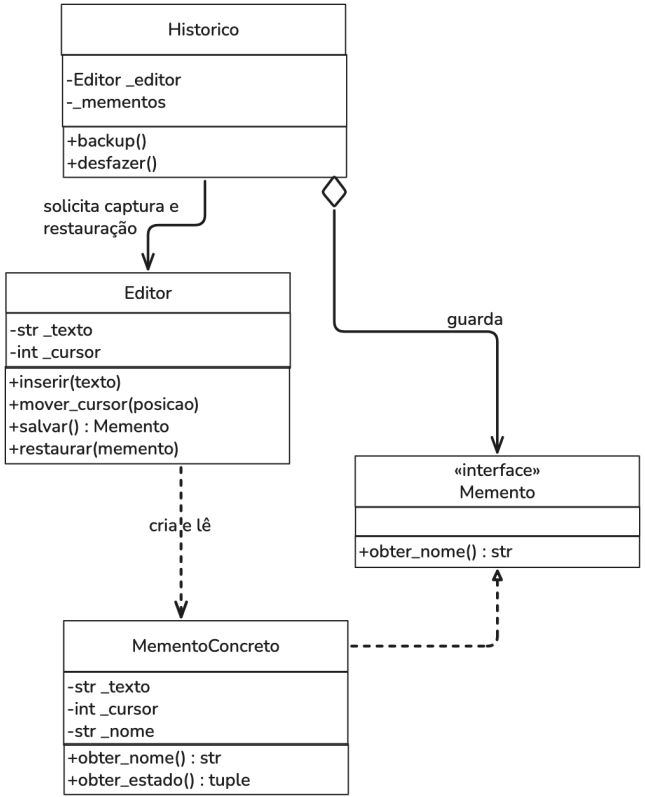
Memento - Estrutura baseada em uma interface intermediária

1. Na ausência de classes aninhadas, você pode restringir o acesso aos campos do memento ao estabelecer uma convenção para que cuidadoras possam trabalhar com um memento através apenas de uma interface intermediária explicitamente declarada
2. As originadoras podem trabalhar com um objeto memento diretamente, acessando campos e métodos declarados na classe memento



Aplicando ao nosso contexto





Aplicando ao nosso contexto - Memento

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass


class Memento(ABC):
    @abstractmethod
    def obter_nome(self) -> str:
        pass


@dataclass(frozen=True)
class MementoConcreto(Memento):
    _texto: str
    _cursor: int
    _nome: str = "Antes da edição"

    def obter_nome(self) -> str:
        return self._nome

    def obter_estado(self) -> tuple[str, int]:
        return self._texto, self._cursor
```

- `frozen=True` impede a reatribuição dos campos após a criação
- `obter_estado()` será utilizado pela Originadora, não pela Cuidadora

Aplicando ao nosso contexto - Originadora (#1)

```python
class Editor:
    def __init__(self, texto: str = "") -> None:
        self._texto = texto
        self._cursor = len(texto)

    @property
    def texto(self) -> str:
        return self._texto

    @property
    def cursor(self) -> int:
        return self._cursor

    def mover_cursor(self, posicao: int) -> None:
        if not 0 <= posicao <= len(self._texto):
            raise ValueError("Posição inválida.")
        self._cursor = posicao

    def inserir(self, texto: str) -> None:
        p = self._cursor
        self._texto = self._texto[:p] + texto + self._texto[p:]
        self._cursor += len(texto)
```

- O Editor mantém o texto e o cursor
- Inserir um texto também atualiza a posição do cursor

Aplicando ao nosso contexto - Originadora (#2)

```python
    def salvar(self) -> Memento:
        return MementoConcreto(self._texto, self._cursor)

    def restaurar(self, memento: Memento) -> None:
        if not isinstance(memento, MementoConcreto):
            raise TypeError("Memento incompatível.")
        self._texto, self._cursor = memento.obter_estado()
```

- `salvar()` captura os valores atuais em um novo retrato
- `restaurar()` recupera os valores desse retrato
- O próprio Editor conhece os dados necessários para realizar essas operações

Aplicando ao nosso contexto - Cuidadora

```python
class Historico:
    def __init__(self, editor: Editor) -> None:
        self._editor = editor
        self._mementos: list[Memento] = []

    def backup(self) -> None:
        self._mementos.append(self._editor.salvar())

    def desfazer(self) -> None:
        if self._mementos:
            self._editor.restaurar(self._mementos.pop())
```

- `backup()` solicita e guarda um retrato.
- `desfazer()` remove o último retrato e o devolve ao editor.
- Se a pilha estiver vazia, o editor permanece como está.

Aplicando ao nosso contexto - Execução

```
editor = Editor("Olá")
historico = Historico(editor)
print(editor.texto, editor.cursor)

historico.backup()
editor.inserir(" mundo")
print(editor.texto, editor.cursor)

historico.backup()
editor.inserir("!")
print(editor.texto, editor.cursor)

historico.desfazer()
print(editor.texto, editor.cursor)

historico.desfazer()
print(editor.texto, editor.cursor)
```

Prós e Contras do Memento

Prós

- O próprio objeto controla a captura e a restauração do seu estado
- A Cuidadora administra o histórico sem interpretar o estado armazenado

Contras

- Muitos retratos podem consumir bastante memória
- É necessário definir quando descartar retratos obsoletos

Use

- Quando for necessário restaurar um estado anterior de um objeto
- Quando copiar atributos por fora do objeto expuser detalhes da sua implementação
- Quando o histórico precisar ser administrado separadamente das regras do objeto

Template Method - O Problema

- Temos uma aplicação para mineração de dados em documentos corporativos.
- Começamos com arquivos DOC. Depois CSV e PDF.

- Só os dois primeiros passos variam. O processo e a análise são idênticos.
- O cliente se torna apenas um condicional

```
if caminho.endswith(".doc"): minerador = MineradorDOC()
elif caminho.endswith(".csv"): minerador = MineradorCSV()
elif caminho.endswith(".pdf"): minerador = MineradorPDF()
```

```
class MineradorDOC:
    def minerar(self, caminho):
        arquivo = self._abrir_doc(caminho)
        bruto = self._extrair_doc(arquivo)
        dados = self._parsear(bruto)
        analise = self._analisar(dados)
        self._enviar_relatorio(analise)
        arquivo.close()
        return analise

class MineradorCSV:
    def minerar(self, caminho):
        arquivo = self._abrir_csv(caminho)
        bruto = self._extrair_csv(arquivo)
        dados = self._parsear(bruto)
        analise = self._analisar(dados)
        self._enviar_relatorio(analise)
        arquivo.close()
        return analise

class MineradorPDF:
    def minerar(self, caminho):
        arquivo = self._abrir_pdf(caminho)
        ...
```

Template Method - A ideia

- Quebre o algoritmo em passos, escreva a sequência uma vez na classe base, e deixe as subclasses redefinirem só os passos que diferem.

```python
class MineradorDeDados(ABC):
    def minerar(self, caminho): # template method
        arquivo = self._abrir(caminho)
        bruto   = self._extrair(arquivo)
        dados   = self._parsear(bruto)
        analise = self._analisar(dados)
        self._enviar_relatorio(analise)
        self._fechar(arquivo)
        return analise
```

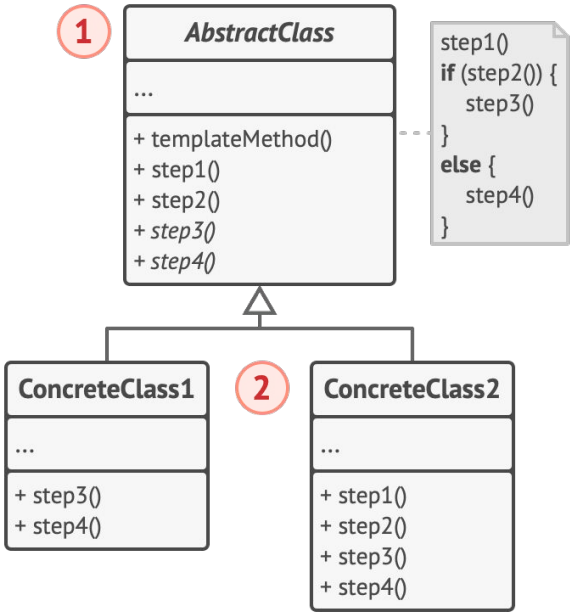
Template Method - Intenção

"Definir o **esqueleto** de um algoritmo em uma operação, postergando a definição de alguns passos para subclasses. O Template Method permite que subclasses redefinam certos passos de um algoritmo sem mudar sua estrutura."

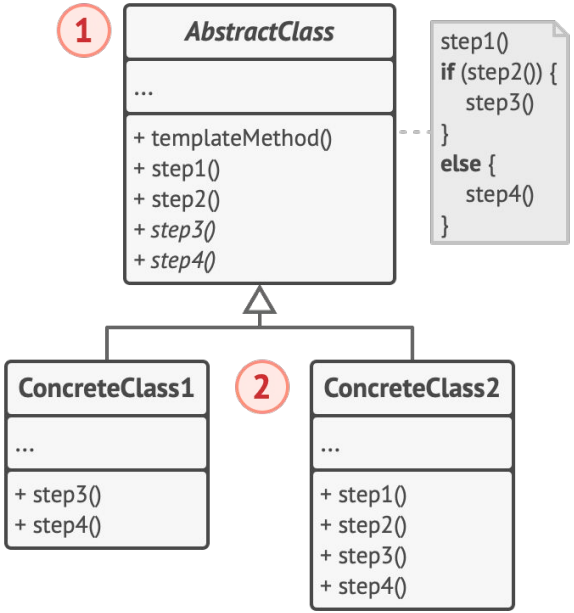
- Padrão comportamental
- Esqueleto é a base que controla a ordem
- As subclasses controlam o conteúdo de passos específicos

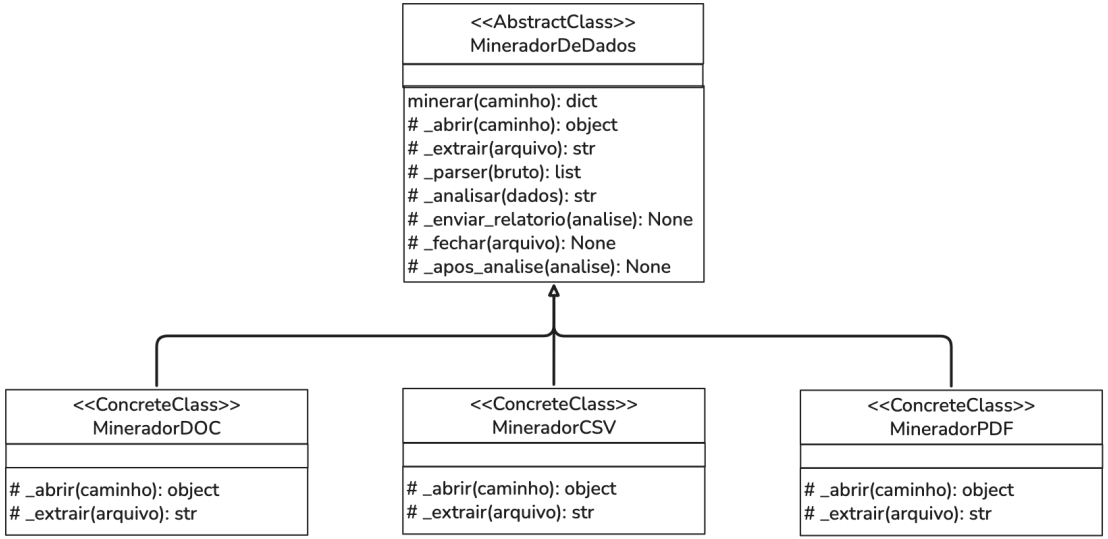
Template Method - Estrutura

1. **AbstractClass** declara métodos que agem como etapas de um algoritmo, bem como o próprio método padrão *(template)* que chama esses métodos em uma ordem específica.
2. **ConcreteClasses** podem sobrescrever todas as etapas, exceto o método padrão.

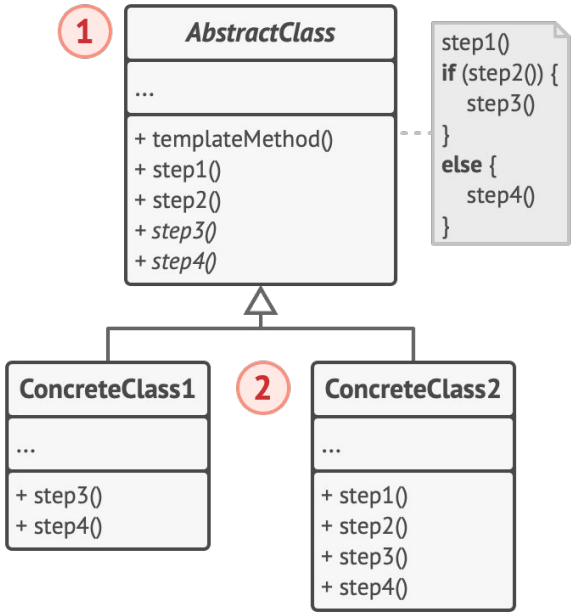


Aplicando ao nosso domínio





Aplicando ao nosso domínio




```python
from abc import ABC, abstractmethod

class MineradorDeDados(ABC):
    def minerar(self, caminho: str) -> dict:
        print(f"Minerando {caminho}")
        arquivo = self._abrir(caminho)
        bruto   = self._extrair(arquivo)
        dados   = self._parsear(bruto)
        analise = self._analisar(dados)
        self._apos_analise(analise)
        self._enviar_relatorio(analise)
        self._fechar(arquivo)
        return analise

    @abstractmethod
    def _abrir(self, caminho: str): ...

    @abstractmethod
    def _extrair(self, arquivo) -> str: ...

    def _parsear(self, bruto: str) -> list:
        return [linha for linha in bruto.split(";") if linha]

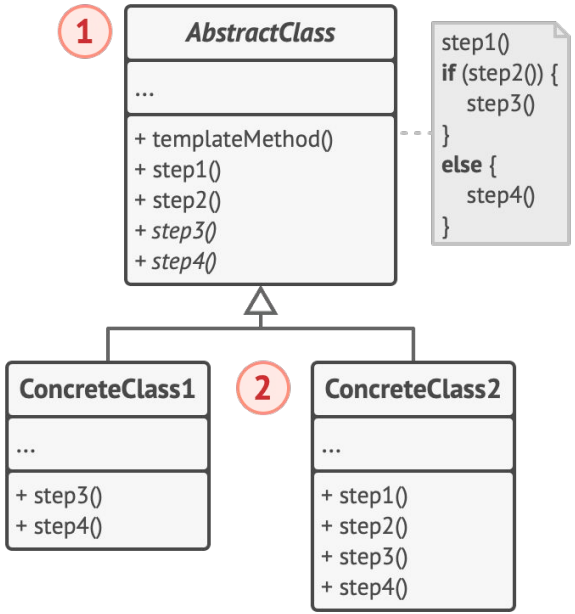
    def _analisar(self, dados: list) -> dict:
        return {"registros": len(dados), "confianca": 1.0}

    def _apos_analise(self, analise: dict) -> None:
        pass

    def _enviar_relatorio(self, analise: dict) -> None:
        print(f"  [REL] {analise['registros']} registros analisados")

    def _fechar(self, arquivo) -> None:
        print("  [IO] arquivo fechado\n")
```

Aplicando ao nosso domínio




```python
class MineradorCSV(MineradorDeDados):
    def _abrir(self, caminho):
        print("  [IO] abrindo em modo texto")
        return f"<csv: {caminho}>"
    def _extrair(self, arquivo):
        return "linha1;linha2;linha3"

class MineradorPDF(MineradorDeDados):
    def _abrir(self, caminho):
        print("  [IO] abrindo em modo binário")
        return f"<pdf: {caminho}>"
    def _extrair(self, arquivo):
        print("  [OCR] extraindo texto")
        return "pagina1;pagina2"
    def _analisar(self, dados):
        return {"registros": len(dados), "confianca": 0.87}
    def _apos_analise(self, analise):
        print(f"  [OCR] confiança média: {analise['confianca']:.0%}")
```

Aplicando ao nosso domínio

```python
for m in (MineradorCSV(), MineradorPDF()):
    m.minerar("relatorio_q3")


# Minerando relatorio_q3
#  [IO] abrindo em modo texto
#  [REL] 3 registros analisados
#  [IO] arquivo fechado

# Minerando relatorio_q3
#  [IO] abrindo em modo binário
#  [OCR] extraindo texto
#  [OCR] confiança média: 87%
#  [REL] 2 registros analisados
#  [IO] arquivo fechado
```

Prós e Contras do Template Method

Prós

- Elimina duplicação, uma vez que o algoritmo mora num lugar só
- O cliente pode variar partes de um algoritmo grande.
- Hooks dão pontos de extensão sem exigir que a subclasse saiba da estrutura.
- É a base de alguns frameworks como unittest, Django, Scrapy spiders.
- Pode ser mais rápido que construir, quando há cálculo caro envolvido.

Contras

- Baseado em herança então a subclasse fica presa à classe mãe.
- Viola o **L** do SO**L**ID caso uma subclasse suprima algo da classe mãe de modo incompatível.
- É mais difícil mudar em tempo de execução.

Não use:

- Se os algoritmos forem muito grandes ou se a classe mãe for se tornar muito extensa.



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br